

Application Note AN 72:

Adding Hi-Speed USB 2.0 To Mobile Phones (w/ PXA27x)



Date: 06/9/2005

1 Applicability

This application note describes the connection from an Intel PXA27x CPU, PLX's NET2272 USB 2.0 Controller, and a mobile phone car kit interface.

Intel PXA27x Multimedia processor family
PLX NET2272REV1A-BC F: USB 2.0 Hi-Speed Peripheral Controller
CEA-936-A: Mini USB analog carkit interface

2 General Description

Figure 1 shows a block diagram of this design and possible USB connections to this design. This configuration provides a Hi-Speed USB Peripheral connection along with a Full-Speed USB Host connection for a mobile handset.

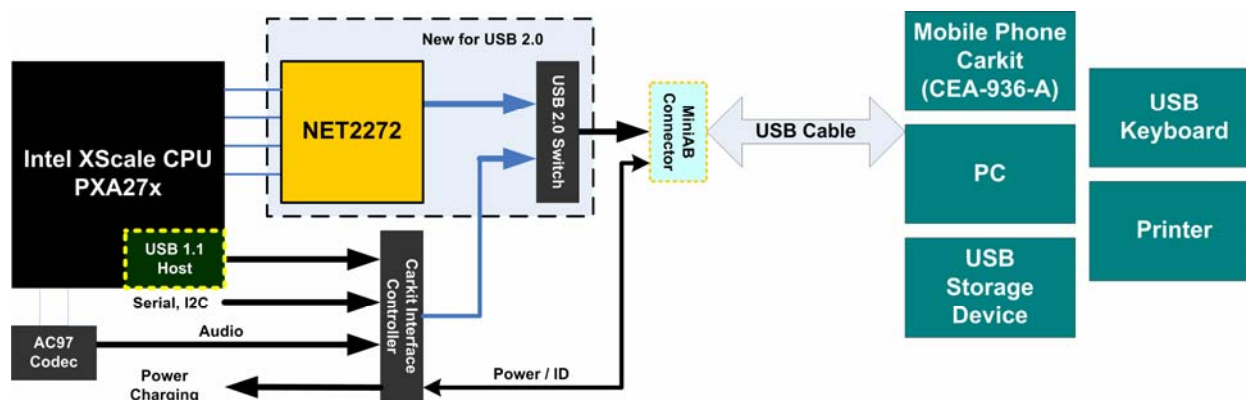


Figure 1: Block Diagram

Notes:

1. The carkit interface controller is needed to meet CEA-936-A requirements. Other car kit standards may not require this controller as CEA-936-A specifies many different signals to be multiplexed onto the same USB pins.
2. The AC97 codec is to interface digital AC97 signals to generate analog audio signals.
3. The USB 2.0 switch (Fairchild FSUSB20) is used to provide minimum edge and phase distortion on the Hi-Speed (480 Mbps) signals. It switches inputs automatically depending on the voltage level of ID pin on the mini-AB connector.
4. The carkit interface controller (Freescale MC13883) also provides power controls for charging the phone battery from the USB wire and providing VBUS power to connected USB devices.

© PLX Technology, Inc., 2005
870 W. Maude Avenue, Sunnyvale, CA 94085
TEL (408) 774-9060 FAX (408) 774-2169
<http://www.plxtech.com>

5. The blue box shows the new components needed for adding a USB Hi-Speed Peripheral Connection. All other components are still required for Full-Speed USB operations.

The car kit used in this application note is the CEA-936-A interface. It is a standard for multiplexing audio (and other) signals over a standard 5-pin mini USB connector between a cell phone and a car kit. The carkit acts like a docking cradle built into a car. Figure 2 shows this interface.

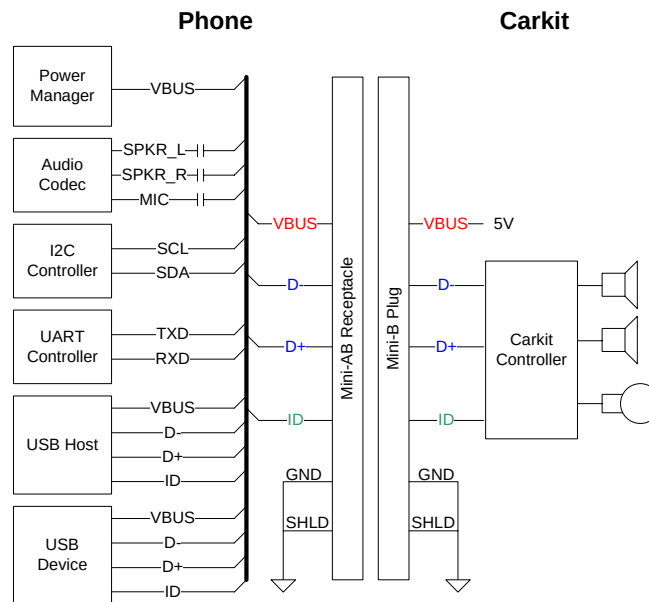


Figure 2: Mini USB Analog Carkit Interface

3 Software Considerations

3.1 Mobile Phone as a Virtual COM device (Serial)

Legacy mobile phones have utilized RS232 serial ports in the past to connect to the PC. To leverage existing PC software built for serial ports, a serial interface over the USB connection is desirable. The USB Communications Device Class (CDC) and host driver offers such an interface. PLX provides a CDC Serial software example that will allow any NET2272-enabled device to enumerate and appear to the USB Host as a serial port. This sample is portable to any platform, including the PXA27x. The CDC host drivers are shipped standard with Windows.

3.2 Mobile Phone as a Network device (RNDIS)

Many mobile phones, like Microsoft SmartPhones, use Remote NDIS (RNDIS) to connect over USB. RNDIS is a protocol specification established by Microsoft which treats the USB connection between the USB host and device as a network interface. The RNDIS host driver, also developed by Microsoft, is available for Windows 2000, XP and all future Windows versions. This connection enables data synchronization through ActiveSync.

3.3 Windows CE based Phones and PDAs

In Windows CE 5.0, Microsoft introduced the USB function (device) controller software stack that has simplified software development for Windows CE USB devices. Programming details for the NET2272 are isolated to a layer called the PDD. A middle layer, called the MDD, serves as an abstraction layer

© PLX Technology, Inc., 2005
870 W. Maude Avenue, Sunnyvale, CA 94085
TEL (408) 774-9060 FAX (408) 774-2169
<http://www.plxtech.com>

between the PDD and a high level USB function client driver. The client driver can use the application programming interface (API) provided by the MDD to implement a class specification, such as Mass Storage Class or any other class or vendor specific protocol specification.

PLX provides a PDD for the NET2272 that is portable to any Windows CE 5.0 platform. Microsoft has developed their own PDD for the NET2272 on the XScale platform that has not been released in the general Windows CE 5.0 development distribution (contact your Microsoft partner for the availability of this driver for your project). Either of these drivers can be used to leverage an existing client driver such as the Mass Storage client driver or and the RNDIS client driver from Windows CE 5.0.

For Windows CE based SmartPhones and PDAs, a RNDIS USB function client driver is provided in Windows CE 5.0. This, combined with the NET2272 PDD (from PLX or Microsoft) provides a near turn-key USB 2.0 High Speed solution for WinCE based phones.

3.4 Linux Support

The Linux community introduced a USB device controller software stack into the Linux kernel at version 2.4.23. This stack defines the Linux USB Gadget API which abstracts the USB device controller from client drivers, such as MassStorage. More information on the Gadget interface is available on the web (<http://www.linux-usb.org/gadget/>). Client drivers for MassStorage, RNDIS and Serial are available in the Linux kernel as well.

PLX provides NET2272 Gadget drivers for Linux kernel versions 2.4 or 2.6. Each is portable to any Linux platform.

3.5 Portable Media and Digital Rights Management (MTP)

The demand for rich media content in the portable device market is growing every day. And with this demand comes the need to protect the content with digital rights management (DRM). A vital part of Microsoft's solution to this need is the Media Transfer Protocol (MTP). As opposed to Mass Storage, which gives a USB Host full control the storage medium, MTP is an object based protocol, which is implemented as a vendor specific class on USB. MTP provides a framework that enables a more sophisticated DRM solution. MTP support is required for Microsoft PlaysforSure Certification.

PLX supplies a MTP user mode client to the GadgetFS interface (Linux). GadgetFS provides a file I/O type interface to the USB device controller. So, user-mode applications can use standard file operations to perform USB transfers on the device endpoints.

PLX's MTP user-mode client will:

- cause the device to enumerate as a MTP device
- handle the MTP transfer state machine over the Bulk endpoints
- handle the MTP commands
- store and retrieve objects from the database (except for files)

Files are stored as file names with the complete path to the file in a database. When the Host performs an operation on a file (Get or Set), the user mode application will open the file and perform the operation on the file (read or write) outside of the database.

The NET2272 has been designed into a myriad of MTP devices, most notably Microsoft's Portable Media Center (which runs WinCE). Furthermore, Microsoft's MTP Device Simulator Kit ships with code that runs on the NET2272. Microsoft's MTP Device Simulator Kit is available here:

<http://www.microsoft.com/downloads/details.aspx?FamilyID=7976db5d-afa4-4dff-83bd-3f83635d2d5b&displaylang=en>

© PLX Technology, Inc., 2005
870 W. Maude Avenue, Sunnyvale, CA 94085
TEL (408) 774-9060 FAX (408) 774-2169
<http://www.plxtech.com>

The NET2272 also supports standard mass storage protocols for WinCE, Linux, and other operating systems if DRM is not desired. For other OS or non-OS platforms, PLX provides generic NET2272 firmware APIs. PLX offers a MassStorage client that interfaces to these firmware APIs. The upper part of the MassStorage client may need to be hooked up to a hard disk in platform-specific ways. A porting guide is available to walk through the establishment of read/write access to the NET2272 and responses to NET2272 interrupts.

4 NET2272 Power Consumption

The NET2272 consumes almost no power when it is placed into suspend state, when all clocks are shut off completely. Table 4 lists the typical power consumption of the NET2272 in different states when 2.5V I/O is used.

NET2272 States	I/O=Core=2.5V	USB_XCVR=3.3V	Total Power
Suspend	0.2 uA	0.08 uA	0.76 uW
Idle	37.7 mA	0.08 uA	94.51 mW
Active	55.5 mA	5.46 mA	156.77 mW

Table 4: NET2272 Power Specifications for 2.5V I/O

Notes:

1. Suspend is when the NET2272 is placed in low power mode and there is no USB connection.
2. Idle is when the NET2272 is running but there is no USB connection and bus is idle.
3. Active is when the NET2272 is connected to USB in Hi-Speed mode with data transferring.

When NET2272's I/O buffers are powered by 1.8V, it requires more considerations to prevent leakage current flowing from 2.5V core power during suspend. Table 5 lists *preliminary* data for power consumption of NET2272 in different states when 1.8V I/O is selected.

NET2272 States	I/O=1.8V	Core=2.5V	USB_XCVR=3.3V	Total Power
Suspend	0.07 uA	0.16 uA	0.08 uA	0.79 uW
Idle	466.7 uA	36.6 mA	0.08 uA	92.34 mW
Active	700 uA	53.7 mA	5.46 mA	153.53 mW

Table 5: NET2272 Power Specifications for 1.8V I/O

Notes:

1. Suspend is when the NET2272 is placed in low power mode and there is no USB connection.
2. Idle is when the NET2272 is running but there is no USB connection and bus is idle.
3. Active is when the NET2272 is connected to USB in Hi-Speed mode with data transferring.

To enter the suspend state, the PXA27x should write a 1 to the **IRQSTAT1** register *Suspend Request Interrupt* bit. When in the suspend state, all inputs to NET2272 including data bus and address bus should not be left floating to prevent un-necessary leakage current on the 2.5V core power supply. It is recommended to park all input and bidirectional signals of NET2272 to high (VDDIO) to minimize power.

The NET2272 automatically returns to the active state when it is re-connected to the USB (cable is plugged in and *USB Detect Enable* bit has been set).

Another way to put the NET2272 into the suspend state is for the PXA27x to drive the CS# pin/ball low while RESET# is asserted (low). To return to idle, the PXA27x drives the CS# pin/ball high and continues

to assert RESET# for a minimum of 2 ms, then de-assert. This suspend state is controlled by the CPU so the NET2272 will not automatically return to active upon re-connection if suspended in this manner.

Please refer to the chapter 7 of the NET2272 data book (reference [1]) for more power management details.

5 Detailed Hardware Description

Following is a table showing the connections between the PXA27x and NET2272 (See Section 4 for the schematic):

Signal	PXA27x	NET2272
Address	MA[7:3]	LA[4:0]
Data	MD[15:0]	LD[15:0]
Chip Select	nCS[1]#	CS#
Read Strobe	nOE	IOR#
Write Strobe	nPWE	IOW#
Interrupt	GPIO[x]	IRQ#

Table 1: Connections between the PXA27x and NET2272

The PXA27x interfaces to the NET2272 on its variable-length I/O (VLIO) bus, using 16-bit, non-multiplexed I/O Read and Write accesses to NET2272 Registers. In 16-Bit mode, the Configuration registers of NET2272 are accessed one byte at a time; however, the buffers of endpoint data are accessed one word at a time.

To support DMA transfers, NET2272 registers must be aligned to 8-byte boundaries in PXA27x memory space. This is done by connecting NET2272's LA[4:0] to PXA27x's MA[7:3]. This can be accounted for by adjusting chip access structures. You can think of the NET2272 memory map as an array of 8 byte structures. The first 2 bytes of each structure map to actual NET2272 registers. The following are example chip access macros:

```
typedef struct NET2272_REG_  
{  
    USHORT Reg;  
    USHORT Reserved1;  
    ULONG Reserved2;  
} NET2272_REG; PNET2272_REG;
```

```
PNET2272_REG NetchipBaseAddress; // Initialized when firmware loads
```

```
#define NETCHIP_READ(ChipReg) NetchipBaseAddress[ChipReg].Reg  
#define NETCHIP_WRITE(ChipReg, Val) NetchipBaseAddress[ChipReg].Reg = Val
```

VLIO access timing is program-adjustable in increments of the PXA27x's internal memory controller clock cycle time (CLK_MEM). External wait states are not employed in this design (READY is pulled high). VLIO functionality is described in section 6.5.5 of reference (2) and section 6.4.3.6 of reference (3). VLIO configuration registers are described in section 6.5.3 of reference (3). Finally, VLIO signal timing is specified in section 6.4.6 of reference (4)

This design is based on a CLK_MEM frequency of **208 MHz**. Operation at other frequencies has not been considered.

© PLX Technology, Inc., 2005
870 W. Maude Avenue, Sunnyvale, CA 94085
TEL (408) 774-9060 FAX (408) 774-2169
<http://www.plxtech.com>

DMA Channels on the PXA27x utilize flow-through, memory-to-memory transfers between internal memory and the NET2272 endpoint buffer register (EP_DATA). The NET2272 register address does not increment between successive reads/writes transfers.

DMA Channels on the PXA27x are counter-driven, moving a pre-programmed amount of data immediately upon firmware command. As such the NET2272 DRQ# and DACK signals are not used in this design.

Wait states are programmable within the PXA27x, and are used exclusively in this design. The PXA27x's READY input pin, therefore, is not used and is pulled high.

Figure 3: Example Connections



Product and Company names are trademarks or registered trademarks of their respective holders.

The following table lists the major components used in the schematic.

Component	Manufacturer	Description
NET2272	PLX Technology	USB 2.0 High Speed device controller
PXA272	Intel	Multimedia processor
FSUSB20	Fairchild	USB High Speed Switch
AC97 CODEC	Analog Devices	Audio codec
0603	Littelfuse PGB Series	Optional if 15kV (air discharge) ESD protection is not required <i>*NET2272 I/O's can sustain 2kV (direct discharge), and FSUSB20 I/O's can sustain 4kV (direct discharge)</i>
MC13883	Freescale	Integrated charger, USB On-the-Go Transceiver and carkit interface

Table 2: Component list

Notes:

1. The FSUSB20 is used to multiplex between USB high speed (480 Mbps) differential signals from the NET2272 and other slower signals including USB host signals from the PXA272.
2. The MC13883 is used to detect three ID levels required for carkit, and it provides VBUS power to the connected USB device when the phone is in USB host mode. It also supports battery charging through a USB connector while also multiplexing audio signals, USB host signals, and communication signals.

Table 3 shows the state of the USB signals in different modes.

Mode	USB_SELECT*	ID*	VBUS	D+	D-
USB Device	0, driven from PXA272	High* level detected from MC13883	5V from PC	pull-up from NET2272	N/A
USB Host	1, driven from PXA272	Low* level detected from MC13883	5V from phone	pull-down from MC13883	pull-down from MC13883
Carkit	1, driven from PXA272	Middle* level detected from MC13883	5V from carkit		

Table 3: Multiplex table

*Notes:

1. USB_SELECT is used to control the D+ and D- selection between the NET2272 and the MC13883.
2. The ID pin in a mini-B plug is connected to ground with resistance greater than 100K ohms.
3. The ID pin in a mini-A plug is connected to ground with resistance smaller than 10 ohms.
4. When connected to the carkit, there are three different states on ID pin, which are detected automatically by the MC13883. Figure 4 shows the relative voltage levels on both the phone ID pin, and the carkit ID pin for the various connect, disconnect and interrupt conditions.

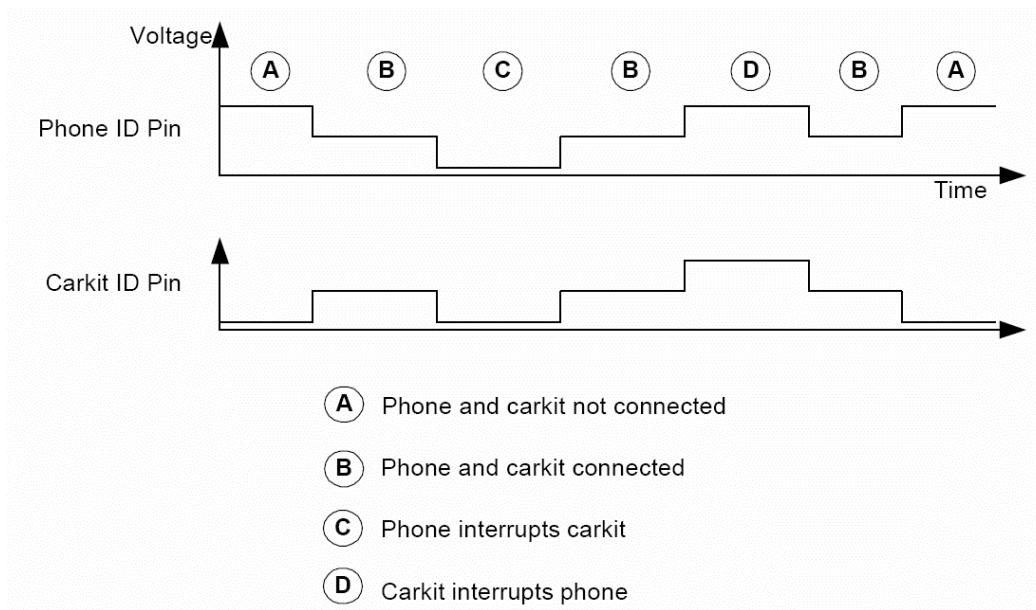


Figure 4: ID Pin Description

7 PXA27x-to-NET2272 Write Access Timing (2.5V I/O*)

PXA27x-to-NET2272 write access timing is detailed in Figure 4. below. NET2272 write timing requirements are detailed in sections 10.4.3.2 and 10.4.4.2 of reference (1). PXA27x VLIO write timing specifications are given in section 6.4.6 of reference (4).

*NET2272 1.8V I/O timings will be released in June, 2005, these diagrams show NET2272 timings with 2.5V I/O.

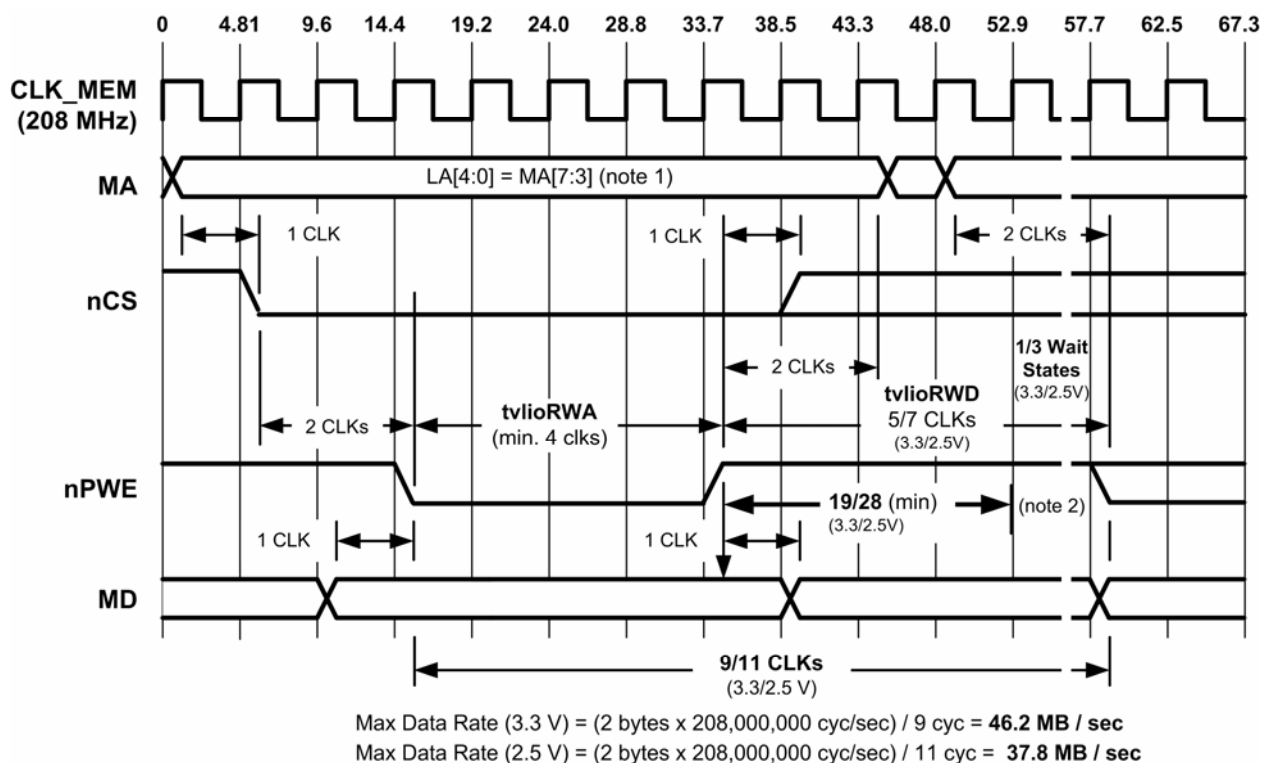


Figure 5: PXA27x-to-NET2272 I/O Write Timing

Notes:

1. NET2272 register addresses are aligned to 8-byte boundaries in PXA27x memory space. MA[2:0] not used.
2. NET2272 requires a minimum delay between nPWE de-assertion and next assertion of 19 and 23 ns, for 3.3V and 2.5V signal levels, respectively. See the NET2272 datasheet for details. To accommodate this requirement, the PXA27x's nPWE high time is programmed to insert 1 or 3 wait states. Wait states are programmed by way of the PXA27x Static Memory Control Register (MSCx), which is described in section 6.5.3.1 of reference (3)
3. The tvlioRWA also specifies nOE high time on VLIO Reads, and tvlioRWD also specifies nOE low time on VLIO reads, such that nPWE and nOE are the inverse of each other timing-wise. As a result, access times for reads and writes are the same.
4. Times of 1 and 2 clocks, where noted in this diagram, are fixed and non-adjustable.
5. READY input of the PXA27x is not used, and is always high. For this reason, it is not shown.

8 PXA27x-to-NET2272 I/O Read Cycle Timing (2.5V I/O*)

PXA27x-to-NET2272 read access timing is detailed in Figure 5. below. NET2272 read timing requirements are detailed in sections 10.4.3.3 and 10.4.4.3 of reference (1). PXA27x VLIO read timing specifications are given in section 6.4.6 of reference (4).

*NET2272 1.8V I/O timings will be released in June, 2005, these diagrams show NET2272 timings with 2.5V I/O.

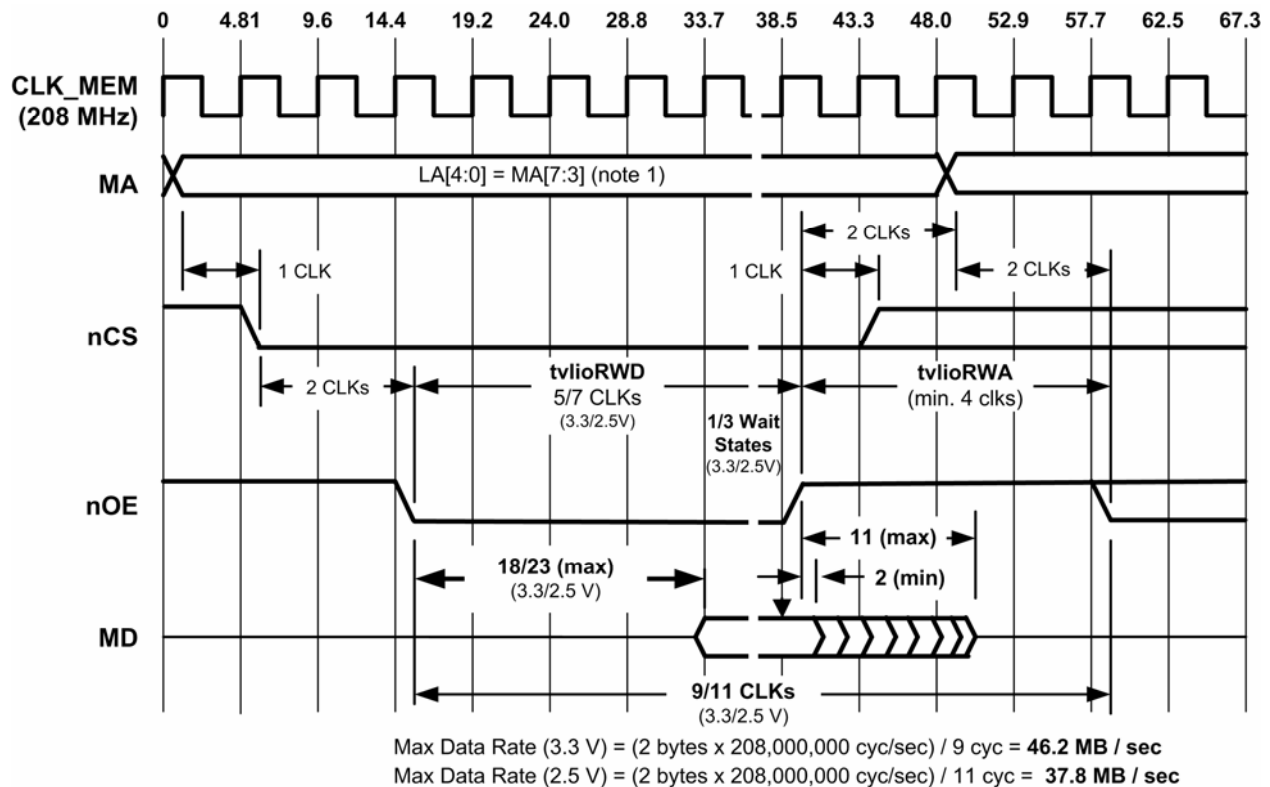


Figure 6: PXA27x-to-NET2272 I/O Read Timing

Notes:

1. NET2272 register addresses are aligned to 8-byte boundaries in PXA27x memory space. MA[2:0] not used.
2. NET2272 requires a minimum delay between nOE assertion and data valid of 18 and 23 ns, for 3.3V and 2.5V signal levels, respectively. See the NET2272 datasheet for details. To accommodate this requirement, the PXA27x's nOE low time is extended by inserting 1 or 3 wait states, for 3.3 or 2.5 volt signaling, respectively. Wait states are programmed by way of the PXA27x Static Memory Control Register (MSCx), which is described in section 6.5.3.1 of reference (3)
3. tvlioRWA also specifies nPWE low time on VLIO writes, and tvlioRWD also specifies nPWE high time on VLIO reads, such that nPWE and nOE are the inverse of each other timing-wise. As a result, access times for reads and writes are the same.
4. Times of 1 and 2 clocks, where noted in this diagram, are fixed and non-adjustable.
5. READY input of the PXA27x is not used, and is always high. For this reason, it is not shown.

9 DMA Transfer Operation

In the PXA27x's conventional DMA modes, fly-by and flow-through modes, external peripheral chips assert DREQ to initiate DMA transfers. Once the DMA transfer has begun, DREQ is to remain asserted for the entire DMA burst. Multiple data transactions occur with each assertion of DREQ.

NET2272 DMA works differently. For USB IN transfers (Device to Host), the NET2272 asserts DREQ# when space is available in the endpoint buffer. When the DMA controller asserts DACK#, the NET2272 latches the data on the data bus, and puts it into the endpoint buffer. Then, the DMA controller is to de-assert DACK# and the NET2272 de-asserts DREQ# (section "10.4 AC Specifications" in the NET2272 datasheet specifies the minimum timing of DREQ# and DACK#). One data transaction occurs for one assertion of DREQ#. This is true for the OUT transfer direction as well. Therefore, the PXA27x's fly-by and flow-through modes cannot be used with the NET2272.

However, the PXA27x does provide a flow-through, memory-to-memory move, DMA mode that can be used with the NET2272. In this mode, the PXA27x DMA controller can be programmed to read or write the NET2272's endpoint buffers using normal programmed I/O cycles. So, if the DMA target is system memory and the DMA source is the NET2272, the PXA27x will read a block of data from the NET2272 using CS# and IOR#, and then write that data to system memory. Since the NET2272 endpoint buffers can only be accessed through the EP_DATA register, the source address increment bit in this case must be cleared. Since the NET2272 has no means to indicate whether the endpoint buffer is full or empty during a programmed I/O cycle, DMA transfers must be performed on a packet-by-packet basis. The next sections address detailed theory of DMA operation.

9.1 USB IN Transfers

For USB IN transfers, the PXA27x's DMA channels could be used for moving data from memory to the NET2272 endpoint buffer (flow-through, memory-to-VLIO read). DMA transfers are performed on a packet-by-packet basis. For transfers spanning several packets, it may be possible to chain a series of packets together using a descriptor list.

The basic sequence of events for an IN transfer would be as follows:

1. High-level device firmware initiates an IN transfer by calling a lower-level transfer function.
2. The lower-level transfer function reads EPAVAIL to determine available space in NET2272 endpoint buffer.
3. If there is space for a full packet, firmware programs the PXA27x DMA controller to load the next packet into the NET2272 endpoint buffer.
4. This process repeats until there is either no space remaining in the endpoint buffer or the transfer is complete.
5. If the buffer is full, then firmware enables the DATA_PACKET_RECEIVED interrupt. When the interrupt occurs, the process continues from step 2 above, until the transfer is complete.
6. If the total transfer size is not a multiple of the maximum packet size, then the firmware should write a 0 to EP_TRANSFER after all the transfer data has been written in order to tell the NET2272 to send the data remaining in the endpoint buffer.

9.2 USB OUT Transfers

For USB OUT transfers, the PXA27x's DMA channels would move data in the opposite direction, from NET2272 endpoint buffer to memory (flow-through, VLIO-to-memory write).

The basic sequence of events for an OUT transfer would be as follows:

© PLX Technology, Inc., 2005
870 W. Maude Avenue, Sunnyvale, CA 94085
TEL (408) 774-9060 FAX (408) 774-2169
<http://www.plxtech.com>

1. High-level device firmware initiates an OUT transfer by calling a lower-level transfer function.
2. Firmware reads the NET2272's EPAVAIL register to determine the number of bytes received from the USB Host in the endpoint buffer.
3. Firmware programs the PXA27x's DMA channel for the transfer, which transfers the received data from the NET2272 endpoint buffer to memory.
4. If the received packet is a full-size packet, then the process continues from step 2 above.
5. If the received packet is less than the maximum packet size, then the transfer is complete.
6. If no additional data has been received, and more data is expected, firmware enables DATA_PACKET_RECEIVED interrupt on the NET2272 endpoint.
7. Upon detection of the DATA_PACKET_RECEIVED interrupt, the process continues from step 2.

10 Summary

This application note describes the connection from an Intel PXA27x CPU, PLX's NET2272 USB 2.0 Controller, and a mobile phone car kit interface. With the design example shown in Figure 3, the following functions are supported through a mini-USB port on a mobile phone:

- Act as a USB 2.0 Hi-Speed Peripheral (mass storage or communication devices) to a PC
- Act as a USB Full-Speed Host to other devices such as mass storage devices and keyboards
- Draw current from a PCs or carkit to power the phone and charge its batteries
- Provide output power to other USB peripherals (keyboards or flash memory drives)
- Communicate via I²C or Universal Asynchronous Receiver Transmitter (UART) with Mini-USB analog carkits and with car stereos
- Send analog speaker signals and receive analog microphone signals from car kits or car stereos

The NET2272's local bus can be interfaced directly to the PXA27x without glue logic, at I/O voltages from 1.8Volts to 3.3 Volts. The NET2272 is interfaced to the PXA27x by way of its 16-bit Variable Latency I/O (VLIO) peripheral interface. Through the VLIO interface all NET2272 internal registers and endpoint buffers can be accessed by either the PXA27x's Xscale ® processor core or its 32-channel DMA controller using VLIO read and write accesses. The PXA27x's DMA channels can be utilized to move packets of transmit/receive data between NET2272 endpoint buffers and PXA27x internal/external memory at high transfer rates (up to 46.2 MBytes/sec), leaving the processor free to perform other tasks concurrently. (The transfer rate is calculated based on the timing charts shown in Figure 5 and 6. Lab tests show that the NET2272 can sustain up to 40 MBytes/sec data transfer rate over USB.)

The small packaging of the NET2272 (6x6mm BGA) makes it well-suited for adding Hi-Speed USB to mobile phones. The low power consumption of the NET2272 facilitates the design of these battery-powered devices. Flexible I/O voltages (1.8V to 5.0V) allow the NET2272 to connect gluelessly to the PXA27x family of processors.

With the USB Communications Device Class, Remote NDIS, MTP, and Mass Storage drivers packaged with Windows, PCs can easily use mobile phones, connected through USB, to setup a high speed wireless internet connection or to transfer files. PLX provides device firmware supporting all of these standard classes.

For further technical support and the latest updates, please contact PLX at:
www.plxtech.com/support

© PLX Technology, Inc., 2005
870 W. Maude Avenue, Sunnyvale, CA 94085
TEL (408) 774-9060 FAX (408) 774-2169
<http://www.plxtech.com>

11 References

1. **NET2272 USB2.0 Peripheral Controller Data Book**, Version 1.6, April 2005, PLX Technology, Inc., <http://www.plxtech.com/products/NET2000/NET2272/default.asp>
2. **Intel®PXA27x Processor Family Design Guide**, March 2004, Intel Corporation, <http://developer.intel.com/design/pca/applicationsprocessors/manuals/28000101.pdf>
3. **Intel® PXA27x Processor Family Developer's Manual**, October 2004, Intel Corporation, <http://developer.intel.com/design/pca/applicationsprocessors/manuals/27882705.pdf>
4. **Intel® PXA27x Processor Electrical, Mechanical, and Thermal Specification**, October 2004, Intel Corporation, <http://developer.intel.com/design/pca/applicationsprocessors/datashts/28000303.pdf>
5. **CEA Standard, Mini USB Analog CarKit Interface Specification, CEA-936**, December 2002, Consumer Electronics Association, <http://global.ihs.com/>
6. **Universal Serial Bus Class Definitions for Communications Devices**, January 1999, USB-IF, http://www.usb.org/developers/devclass_docs/usbc11.pdf
7. **USB Function Controller Drivers**, November 2004, Microsoft Windows CE 5.0 MSDN, <http://msdn.microsoft.com/library/default.asp?url=/library/en-us/wceddk5/html/wce50conusbfunctioncontrollerdriverdevelopmentconcepts.asp>
8. **Remote NDIS (RNDIS) and Windows**, April 2004, Microsoft WHDC, <http://www.microsoft.com/whdc/device/network/NDIS/rmNDIS.msp>

© PLX Technology, Inc., 2005
870 W. Maude Avenue, Sunnyvale, CA 94085
TEL (408) 774-9060 FAX (408) 774-2169
<http://www.plxtech.com>